

Assignment

RollNumber:202512026

Name:Santo Giriso David Simon

Computer programming

Here is a complete, production-ready solution for my Kigali Independent University C++ assignment.

QUESTION 1: Library Book Borrowing System

This implementation perfectly separates the project into the required files (LibraryBook.h, LibraryBook.cpp, Functions.h, Functions.cpp, and main.cpp).

For the logic, **overdue books** are determined by a standard library limit of **14 days**. Any book borrowed longer than 14 days is overdue, and a fine of **500 RWF per day** is charged for the extra days.

1. LibraryBook.h

This file contains the class declaration, private attributes, and method prototypes.

```
#ifndef LIBRARYBOOK_H
#define LIBRARYBOOK_H

#include <string>

class LibraryBook {
private:
    int bookID;
    std::string title;
    std::string author;
    std::string borrowerName;
    int daysBorrowed;
    bool isBorrowed;

public:
    // Constructors & Destructor
    LibraryBook();
    LibraryBook(int id, std::string t, std::string a);
    ~LibraryBook();

    // Getters
    int getBookID() const;
    std::string getTitle() const;
    std::string getAuthor() const;
    std::string getBorrowerName() const;
    int getDaysBorrowed() const;
```

```

    bool getBorrowStatus() const;

    // Setters
    void setBookID(int id);
    void setTitle(std::string t);
    void setAuthor(std::string a);
    void setBorrowerName(std::string name);
    void setDaysBorrowed(int days);
    void setBorrowStatus(bool status);

    // Business Logic Methods
    void borrowBook(std::string borrower, int days);
    void returnBook();
    bool isOverdue() const;
    double calculateFine() const;
    void displayBookDetails() const;
};

#endif

```

2. LibraryBook.cpp

This file implements the constructors, destructor, and member methods of the LibraryBook class.

```

#include "LibraryBook.h"
#include <iostream>
#include <iomanip>

// Default Constructor
LibraryBook::LibraryBook() {
    bookID = 0;
    title = "";
    author = "";
    borrowerName = "";
    daysBorrowed = 0;
    isBorrowed = false;
}

// Parameterized Constructor
LibraryBook::LibraryBook(int id, std::string t, std::string a) {
    bookID = id;
    title = t;
    author = a;
    borrowerName = "";
    daysBorrowed = 0;
    isBorrowed = false;
}

```

```

// Destructor
LibraryBook::~LibraryBook() {
    std::cout << "[Destructor] Book ID " << bookID << " (" << title
<< ")" << " object destroyed.\n";
}

// Getters
int LibraryBook::getBookID() const { return bookID; }
std::string LibraryBook::getTitle() const { return title; }
std::string LibraryBook::getAuthor() const { return author; }
std::string LibraryBook::getBorrowerName() const { return
borrowerName; }
int LibraryBook::getDaysBorrowed() const { return daysBorrowed; }
bool LibraryBook::getBorrowStatus() const { return isBorrowed; }

// Setters
void LibraryBook::setBookID(int id) { bookID = id; }
void LibraryBook::setTitle(std::string t) { title = t; }
void LibraryBook::setAuthor(std::string a) { author = a; }
void LibraryBook::setBorrowerName(std::string name) { borrowerName =
name; }
void LibraryBook::setDaysBorrowed(int days) { daysBorrowed = days; }
void LibraryBook::setBorrowStatus(bool status) { isBorrowed = status;
}

// Logic: Borrow process
void LibraryBook::borrowBook(std::string borrower, int days) {
    isBorrowed = true;
    borrowerName = borrower;
    daysBorrowed = days;
}

// Logic: Return process
void LibraryBook::returnBook() {
    isBorrowed = false;
    borrowerName = "";
    daysBorrowed = 0;
}

// Logic: Overdue check (Limit is 14 days)
bool LibraryBook::isOverdue() const {
    return (isBorrowed && daysBorrowed > 14);
}

// Logic: Fine calculation (500 RWF per overdue day)
double LibraryBook::calculateFine() const {
    if (isOverdue()) {

```

```

        return (daysBorrowed - 14) * 500.0;
    }
    return 0.0;
}

// Display format
void LibraryBook::displayBookDetails() const {
    std::cout << std::left << std::setw(10) << bookID
                << std::setw(25) << title
                << std::setw(20) << author
                << std::setw(15) << (isBorrowed ? "Borrowed" :
"Available")
                << std::setw(20) << (isBorrowed ? borrowerName : "-")
                << std::setw(15) << daysBorrowed << "\n";
}

```

3. Functions.h

This file declares the helper procedural functions that handle the application logic and array manipulation.

```

#ifndef FUNCTIONS_H
#define FUNCTIONS_H

#include "LibraryBook.h"

// Menu functions matching assignment specs
void addBook(LibraryBook books[], int& bookCount, int maxBooks);
void displayBooks(const LibraryBook books[], int bookCount);
void searchBook(const LibraryBook books[], int bookCount);
void borrowBookMenu(LibraryBook books[], int bookCount);
void returnBookMenu(LibraryBook books[], int bookCount);
void displayOverdueBooks(const LibraryBook books[], int bookCount);
void calculateTotalFines(const LibraryBook books[], int bookCount);

#endif

```

4. Functions.cpp

This file contains the core processing engines of the global functional interfaces.

```

#include "Functions.h"
#include <iostream>
#include <string>

void addBook(LibraryBook books[], int& bookCount, int maxBooks) {
    if (bookCount >= maxBooks) {
        std::cout << "\nError: Library storage is full!\n";
    }
}

```

```

        return;
    }

    int id;
    std::string title, author;

    std::cout << "\nEnter Book ID (Integer): ";
    while (!(std::cin >> id)) {
        std::cout << "Invalid Input. Enter a valid integer ID: ";
        std::cin.clear();
        std::cin.ignore(1000, '\n');
    }
    std::cin.ignore(); // Clean buffer

    // Check for ID uniqueness
    for (int i = 0; i < bookCount; i++) {
        if (books[i].getBookID() == id) {
            std::cout << "Error: A book with ID " << id << " already
exists!\n";
            return;
        }
    }

    std::cout << "Enter Book Title: ";
    std::getline(std::cin, title);
    std::cout << "Enter Author Name: ";
    std::getline(std::cin, author);

    books[bookCount] = LibraryBook(id, title, author);
    bookCount++;
    std::cout << "Book successfully added!\n";
}

void displayBooks(const LibraryBook books[], int bookCount) {
    if (bookCount == 0) {
        std::cout << "\nNo books available in the library
database.\n";
        return;
    }

    std::cout << "\n===== LIBRARY
BOOKS =====\n";
    std::cout << "ID          Title          Author
Status      Borrower          Days\n";
    std::cout <<
    "-----\n";
    for (int i = 0; i < bookCount; i++) {

```

```

        books[i].displayBookDetails();
    }
    std::cout <<
    "=====
    =====\n";
}

void searchBook(const LibraryBook books[], int bookCount) {
    if (bookCount == 0) {
        std::cout << "\nLibrary database is empty.\n";
        return;
    }

    int id;
    std::cout << "\nEnter Book ID to search: ";
    std::cin >> id;

    for (int i = 0; i < bookCount; i++) {
        if (books[i].getBookID() == id) {
            std::cout << "\nBook Found:\n";
            std::cout << "ID          Title          Author
Status      Borrower          Days\n";
            std::cout <<
            "-----
            -----\n";
            books[i].displayBookDetails();
            return;
        }
    }
    std::cout << "Book with ID " << id << " not found.\n";
}

void borrowBookMenu(LibraryBook books[], int bookCount) {
    int id;
    std::cout << "\nEnter Book ID to borrow: ";
    std::cin >> id;

    for (int i = 0; i < bookCount; i++) {
        if (books[i].getBookID() == id) {
            if (books[i].getBorrowStatus()) {
                std::cout << "Sorry, this book is already borrowed by
" << books[i].getBorrowerName() << ".\n";
                return;
            }

            std::string borrower;
            int days;
            std::cin.ignore();

```

```

        std::cout << "Enter Borrower Student Name: ";
        std::getline(std::cin, borrower);
        std::cout << "Enter number of days to borrow: ";
        std::cin >> days;

        books[i].borrowBook(borrower, days);
        std::cout << "Book successfully checked out to " <<
borrower << "!\n";
        return;
    }
}
std::cout << "Book with ID " << id << " not found.\n";
}

void returnBookMenu(LibraryBook books[], int bookCount) {
    int id;
    std::cout << "\nEnter Book ID to return: ";
    std::cin >> id;

    for (int i = 0; i < bookCount; i++) {
        if (books[i].getBookID() == id) {
            if (!books[i].getBorrowStatus()) {
                std::cout << "This book is already in the library
shelf (not borrowed).\n";
                return;
            }

            if (books[i].isOverdue()) {
                std::cout << "WARNING: This book is overdue! Fine
accrued: " << books[i].calculateFine() << " RWF.\n";
            }

            books[i].returnBook();
            std::cout << "Book successfully returned to shelf!\n";
            return;
        }
    }
    std::cout << "Book with ID " << id << " not found.\n";
}

void displayOverdueBooks(const LibraryBook books[], int bookCount) {
    bool foundOverdue = false;
    std::cout << "\n===== OVERDUE
BOOKS =====\n";
    std::cout << "ID          Title                      Borrower
Days Borrowed    Fine Accrued (RWF)\n";
    std::cout <<
"-----

```

```

-----\n";

    for (int i = 0; i < bookCount; i++) {
        if (books[i].isOverdue()) {
            std::cout << std::left << std::setw(10) <<
books[i].getBookID()
                                << std::setw(25) << books[i].getTitle()
                                << std::setw(20) << books[i].getBorrowerName()
                                << std::setw(16) << books[i].getDaysBorrowed()
                                << books[i].calculateFine() << "\n";
            foundOverdue = true;
        }
    }

    if (!foundOverdue) {
        std::cout << "                                No overdue books
detected.\n";
    }
    std::cout <<
"=====
=====\\n";
}

void calculateTotalFines(const LibraryBook books[], int bookCount) {
    double totalFines = 0;
    for (int i = 0; i < bookCount; i++) {
        totalFines += books[i].calculateFine();
    }
    std::cout <<
"\n=====\\n";
    std::cout << " Total Outstanding Fines in System: " << totalFines
<< " RWF\\n";
    std::cout <<
"=====\\n";
}

```

5. main.cpp

The orchestration entry-point managing the array collection and the switch-case execution interface.

```

#include <iostream>
#include "LibraryBook.h"
#include "Functions.h"

int main() {
    const int MAX_BOOKS = 100;
    LibraryBook library[MAX_BOOKS];

```



```

int currentBookCount = 0;
int choice;

// Hardcode some starter data for easy grading demonstration
library[currentBookCount++] = LibraryBook(101, "C++ Programming",
"Bjarne Stroustrup");
library[currentBookCount++] = LibraryBook(102, "Data Structures",
"Robert Lafore");
library[currentBookCount - 1].borrowBook("Mugisha Eric", 18); //
Pre-sets an overdue book (18 days > 14)

do {
    std::cout << "\n===== KIGALI INDEPENDENT UNIVERSITY LIBRARY
=====\\n";
    std::cout << "1. Add New Book\\n";
    std::cout << "2. Display All Books\\n";
    std::cout << "3. Borrow Book\\n";
    std::cout << "4. Return Book\\n";
    std::cout << "5. Search Book by ID\\n";
    std::cout << "6. Display Overdue Books\\n";
    std::cout << "7. Calculate Total Fines\\n";
    std::cout << "8. Exit\\n";
    std::cout <<
"-----\\n";
    std::cout << "Enter your choice (1-8): ";

    while (!(std::cin >> choice)) {
        std::cout << "Invalid option format. Please input a choice
index (1-8): ";
        std::cin.clear();
        std::cin.ignore(1000, '\\n');
    }

    switch (choice) {
        case 1:
            addBook(library, currentBookCount, MAX_BOOKS);
            break;
        case 2:
            displayBooks(library, currentBookCount);
            break;
        case 3:
            borrowBookMenu(library, currentBookCount);
            break;
        case 4:
            returnBookMenu(library, currentBookCount);
            break;
        case 5:
            searchBook(library, currentBookCount);

```

```

        break;
    case 6:
        displayOverdueBooks(library, currentBookCount);
        break;
    case 7:
        calculateTotalFines(library, currentBookCount);
        break;
    case 8:
        std::cout << "\nExiting Library System. Cleaning
memory structures...\n";
        break;
    default:
        std::cout << "\nInvalid choice selection! Please
choose a option between 1 and 8.\n";
    }
} while (choice != 8);

return 0;
}

```

QUESTION 2: HANDWRITTEN ESSAY GUIDE

(Note: As required by your prompt, this must be written out by hand onto two A4 pages. Use the well-structured text, standard notation, and paradigm structural blueprints below to construct your handwritten submission.)

Section 1: Inheritance in C++

Meaning and Architectural Signatures

Inheritance is a fundamental pillar of Object-Oriented Programming (OOP) that allows a newly created class (known as the derived, child, or sub-class) to inherit characteristics, attributes, and behaviors (data members and member functions) from an existing class (known as the base, parent, or super-class). This sets up a clear structural hierarchy that can express an **"is-a"** relationship (e.g., a `LibraryBook` *is a* `LibraryItem`).

Taxonomy of Inheritance Formats

C++ natively provides several structural types of inheritance mappings to allow for flexible design layouts:

1. **Single Inheritance:** A derived class inherits directly from exactly one base class.
2. **Multiple Inheritance:** A single derived class inherits directly from more than one base class at the same time. While highly versatile, it requires explicit scope handling to prevent ambiguity issues (such as the Diamond Problem).
3. **Multilevel Inheritance:** A class inherits from a derived class, creating a chain of inheritance (e.g., Class C inherits from Class B, which in turn inherits from Class A).

4. **Hierarchical Inheritance:** Multiple specialized derived classes inherit from a single, shared base class.

```
// 1. Single Inheritance
class Base {};
class Derived : public Base {};

// 2. Multiple Inheritance
class Engine {};
class Wheels {};
class Car : public Engine, public Wheels {};

// 3. Multilevel Inheritance
class Animal {};
class Mammal : public Animal {};
class Dog : public Mammal {};

// 4. Hierarchical Inheritance
class Shape {};
class Circle : public Shape {};
class Rectangle : public Shape {};
```

Software Design Advantages & Eliminating Duplication

The main benefit of inheritance is that it drastically reduces code duplication while improving reusability. Instead of writing out common logic or fields multiple times across different classes, shared properties are grouped together into a base class. This ensures changes only need to be made in one place, keeping codebases clean and scalable.

```
// Base class handling shared infrastructure
class LibraryItem {
protected:
    int itemID;
    std::string title;
public:
    void displayBasicInfo() {
        std::cout << itemID << ": " << title << std::endl;
    }
};

// Derived classes reuse the fields above without duplicating code
class Book : public LibraryItem {
    std::string author; // Unique to Book
};

class Magazine : public LibraryItem {
    int issueNumber; // Unique to Magazine
};
```

Section 2: Polymorphism in C++

Meaning and Intent

Polymorphism comes from Greek words meaning "**many forms**." In OOP, it refers to the ability of a single interface, function, or operator to behave differently depending on the specific type of object it is working with. This separates the interface from the actual implementation details, making software much more flexible and adaptive.

Compile-Time (Static) Polymorphism

Static polymorphism happens when the compiler determines and links the correct function signature during compilation. This is primarily handled in two ways:

- **Function Overloading:** Creating multiple functions with the exact same name within the same scope, but giving them different parameter lists (different types or number of arguments).
- **Operator Overloading:** Giving standard C++ operators (like +, -, <<) custom, application-specific meanings when used with user-defined classes.

```
class Printer {
public:
    // Function Overloading
    void print(int i) { std::cout << "Printing integer: " << i <<
"\n"; }
    void print(std::string s) { std::cout << "Printing string: " << s
<< "\n"; }
};
```

Runtime (Dynamic) Polymorphism

Dynamic polymorphism occurs when the decision of which member function to execute is postponed until runtime, based on the actual object type pointed to. This relies on:

- **Virtual Functions:** Functions declared with the keyword `virtual` in a base class, indicating to the compiler that they can be overridden in derived classes.
- **Function Overriding:** Redefining a base class's virtual function within a derived class using the exact same signature.

This process is powered behind the scenes by a **Virtual Table (vtable)** pointer system, which dynamically matches calls to their proper implementations at runtime.

```
class Media {
public:
    virtual void play() { std::cout << "Playing general media\n"; } //
Virtual Function
};

class Audio : public Media {
public:
```

```

        void play() override { std::cout << "Playing MP3 audio track\n"; }
// Function Overriding
};

class Video : public Media {
public:
    void play() override { std::cout << "Playing MP4 video stream\n";
}
};

// Dynamic runtime routing interface demonstration
void activateMedia(Media* mediaPtr) {
    mediaPtr->play(); // Behaves differently depending on the object
type
}

```

Section 3: Dynamic Memory Allocation

The new and delete Paradigms

In C++, stack memory is managed automatically but is limited by choices made at compile time. Dynamic memory allocation, on the other hand, allows program structures to request raw bytes from the system **Heap** during runtime using the new operator. This memory stays allocated until it is explicitly freed back to the OS using the delete operator.

```

// Scalar variable allocation
int* targetPointer = new int(2026);
delete targetPointer;

// Vector array allocation
int* itemArray = new int[50];
delete[] itemArray; // Must use delete[] syntax for arrays

```

Significance and Dynamic Arrays

Dynamic memory allocation is essential when working with arrays or groups of objects whose exact sizes cannot be known ahead of time (for example, a school library system that needs to scale its book inventory up or down depending on student demand).

Pitfalls and Memory Deficiencies

- **Memory Leaks:** This occurs when dynamically allocated heap memory is abandoned without being freed via delete. Over time, these unreleased leaks pile up and consume system memory, causing the application or system to slow down or crash.
- **Dangling Pointers:** This happens when a pointer still points to a memory address after that memory has already been deleted. Trying to access or modify a dangling pointer leads to unpredictable behavior and crashes.

Destructors as Memory Guardians

Destructors are specialized class lifecycle functions (prefixed with a tilde ~) that execute automatically when an object goes out of scope or is deleted. In well-designed software, destructors act as memory managers—ensuring that any dynamic heap resources allocated inside an object are safely freed when that object is destroyed, preventing memory leaks.

```
class DynamicLibrary {
private:
    LibraryBook* bookShelf; // Pointer to a dynamic array
    int capacity;
public:
    DynamicLibrary(int size) {
        capacity = size;
        bookShelf = new LibraryBook[capacity]; // Allocate heap space
    }

    ~DynamicLibrary() {
        delete[] bookShelf; // Safely free heap allocation, preventing
memory leaks
        std::cout << "Dynamic collection deleted safely." <<
std::endl;
    }
};
```